Initiation à la programmation avec le langage C

Table des matières

I. Introduction : 3
1) Comment programmer ? 3
2) La compilation 3
3) La notion d'Expression & d'Instruction 3
II. Le bases du langage C 4
1) Structure d'un programme C 4
2) Les bibliothèques 5
3) Les composants élémentaires du C 5
4) Les types prédéfinis : 6
a. Type caractère 6
b. Les types entiers 7
c. Les types réels 8
5) Déclarations des variables 8
6) Les constantes 8
7) Les opérateurs 9
8) La fonction principale main 10
9) Exercice 10
III. Le dialogue avec l'utilisateur 10
1) La fonction d'écriture printf: 10
2) Exercices 12
IV. Les instructions Alternatives 12
1) L'instruction if 13
2) Branchement multiple switch 13
V. Les instructions répétitives 13
1) Boucle while : 13
2) Boucle do---while 14
3) Boucle for 14
4) Exercices 14

I. Introduction :

1) Comment programmer ?

Résoudre un problème

- Penser son code (raisonnement logique et intuitif) Détailler les étapes complètes du programme à concevoir - développer l'algorithme correspondant
- Implémenter (contrôler et commenter chaque étape) Ecrire (traduire) l'algorithme dans le langage de programmation choisi (C/C++)
- Compiler / Exécuter : Transformer le code en un programme exécutable par l'ordinateur
- Debugger si nécessaire (retour au point 1)

2) La compilation

Le C est un langage compilé : Cela signifie qu'un programme C est décrit par un fichier texte, appelé fichier source.
Ce fichier n'étant évidemment pas exécutable par le microprocesseur, il faut le traduire en langage machine via un compilateur :
Syntaxe de compilation d'un programme C :

gcc [options] fichier.c [-llibrairies]

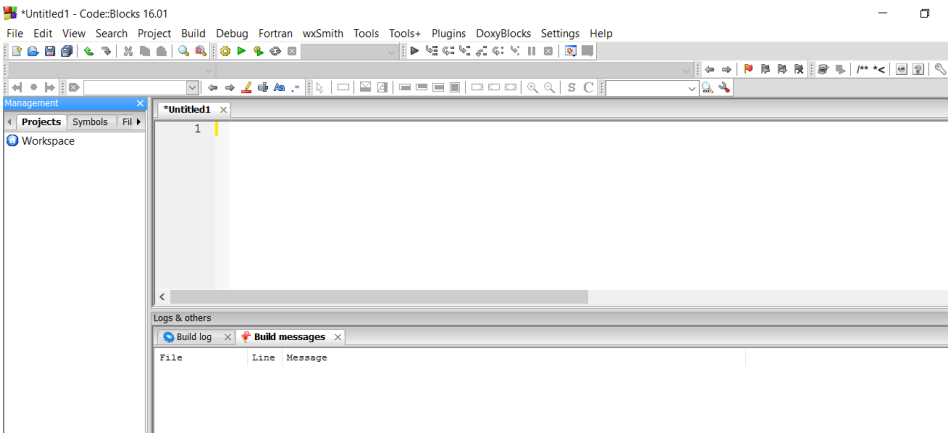

Figure 1 : Code::Blocks

3) La notion d'Expression & d'Instruction

Une expression est une suite de composants élémentaires syntaxiquement correcte, par exemple :

x = 2 ou bien (i >= 0) && (i < 10) && (p[i] != 0)

Une *instruction est une expression suivie d'un point-virgule qui signifie « é*valuer cette expression », Plusieurs instructions peuvent être rassemblées par des accolades *{ et }* par exemple :

```
if (x != 0) {
        z = y / x;
        t = y % x;
}

int a;   //une instruction qui a pour objectif de déclarer la variable a
 int b = 1, c;
```

II. Le bases du langage C

1) Structure d'un programme C

En C, toute variable doit faire l'objet d'une déclaration avant d'être utilisée.
Un programme C se présente de la façon suivante :

- [directives au préprocesseur]
- [déclarations de variables externes]
- [fonctions secondaires]
- [fonction principale]

```
/* des inclusions de fichiers en-tete (fonctions dans des bibliothèques) */
#include <nom_bibliotheque>


/* des définitions de variables et constantes */
definition de constante_1 ;
...
definition de constante_M ;
definition de variable_1 ;
...
definition de variable_N ;

/* des fonctions */
definition de la fonction 1
{
   corps de la fonction 1
}
...

definition de la fonction P
{
   corps de la fonction P
}


/* partie principale */
definition de la fonction principale
{
  corps de la fonction principale
}
```

Figure 2 : Structure d'un programme C

2) Les bibliothèques

En C comme en d'autres langages, il existe des bibliothèques qui contiennent des fonctions déjà pré-défini d'entrées/sorties telles que printf, scanf, des fonctions pour la manipulation de chaînes de caractères strlen, strcpy...

Utiliser des bibliothèques : Lorsqu'on compile, on doit indiquer quelles sont les bibliothèques utilisées par le programme.

Utilisé par l'expression :

#include <nom-de-fichier>

Exemple : #include <stdio.h> //bibliothéque des entrées et sorties

Utilisé par l'expression :

- La gestion des fichiers (**stdio.h**) /* Entrees-sorties standard */
- Les fonctions mathématiques (**math.h**)
- Traitement de chaînes de caractères (**string.h**)
- Le traitement de caractères (**ctype.h**)
- Utilitaires généraux (**stdlib.h**)
- Date et heure (**time.h**)

3) Les composants élémentaires du C

Un programme en langage C est constitué des groupes de composants élémentaires suivants :

- Les identificateurs : de donner un nom à une entité du programme (nom d'une variable ou de fonction, un type pré défini, etc.)
- Les mots-clefs : sont réservés pour le langage lui-même (if, else, const, return, while, etc.)
- Les commentaires : Un commentaire débute par /* et se termine par */.

Remarque préliminaire : commenter le code !

Veiller toujours à commenter le code ! ! ! De nombreuses lignes de code à la suite les unes des autres deviennent très rapidement illisibles.

→ on ajoute du texte pour expliquer à quoi servent les lignes de code.

Exemple :

```c
#include <stdio.h>
/* Programme pour calculer la moyenne
à partir des notes saisies des etudiants
*/
int main() {
    int nbEtudiant; //Variable pour stocker le nombre des étudiants
    printf("Veuillez préciser le nombre des étudiants"); //demander à l'utilisate
    scanf("%d",&nbEtudiant); //lire le nomnbre des étduaints
    for (i=0;i<nbEtudiant;i++) {
        //saisir la note
        scanf("%d",&note);
    }
}
```

4) Les types prédéfinis :

Le C est un langage typé ➔ toute variable, constante ou fonction est d'un type précis.

La mémoire de l'ordinateur se décompose en une suite continue d'octets. Chaque octet de la mémoire est caractérisé par son adresse.

Quand une variable est définie, il lui est attribué une adresse. Cette variable correspondra à une zone mémoire dont la longueur (le nombre d'octets) est fixée par le type.

Les types de base en C concernent les caractères, les entiers et les flottants (nombres réels).

Ils sont désignés par les mots-clefs suivants :

Char int float double short long unsigned

a. *Type caractère*

Le mot-clef char désigne un objet de type caractère. Un char peut contenir n'importe quel élément du jeu de caractères de la machine utilisée. La plupart du temps, un objet de type char est codé sur un octet ; c'est l'objet le plus élémentaire en C.

Une des particularités du type char en C est qu'il peut être assimilé à un entier : c+1 est valide et désigne le caractère suivant dans le code ASCII.

Exemple :

```c
int main() {
        char c = 'A';
         printf("%c", c +1 );
}
```

Dec	→	0	16	32	48	64	80	96	112	128	144	160	176	192	208	224	240
↓	Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	nul	□		0	@	P	`	p	Ç	É	á	□	□	□	α	≡
1	1	☺	□	!	1	A	Q	a	q	ü	æ	í	□	□	□	β	±
2	2	□	×	"	2	B	R	b	r	é	Æ	ó	□	□	□	Γ	≥
3	3	♥	!!	#	3	C	S	c	s	â	ô	ú	□	□	□	π	≤
4	4	♦	¶	$	4	D	T	d	t	ä	ö	ñ	□	□	□	Σ	⌠
5	5	♣	§	%	5	E	U	e	u	à	ò	Ñ	□	□	□	σ	⌡
6	6	♠	□	&	6	F	V	f	v	å	û	□	□	□	□	µ	÷
7	7	•	□	'	7	G	W	g	w	ç	ù	□	□	□	□	τ	≈
8	8	□	↑	(	8	H	X	h	x	ê	ÿ	¿	□	□	□	Φ	°
9	9	○	↓	)	9	I	Y	i	y	ë	Ö	□	□	□	□	θ	•
10	A	□	→	*	:	J	Z	j	z	è	Ü	¬	□	□	□	Ω	·
11	B	□	←	+	;	K	[	k	{	ï	¢	½	□	□	□	δ	√
12	C	□	□	,	<	L	\	l	¦	î	£	¼	□	□	□	∞	n
13	D	□	Ö	–	=	M	]	m	}	ì	¥	¡	□	□	□	φ	²
14	E	□	s	.	>	N	^	n	~	Ä	□	«	□	□	□	∈	z
15	F	⌖	t	/	?	O	_	o	•	Å	□	»	□	□	□	∩	□

Figure 3 : Table CODE ASCII

b. *Les types entiers*

Le mot-clef désignant le type entier est int.
Le type int peut être précédé d'un attribut de précision (short ou long) et aussi et/ou d'un attribut de représentation (unsigned)

En général :

- Un short int est codé sur 16 bits.
- Un objet de type long int a au moins la taille d'un int.

En C, on utilisera un des types suivants en fonction de la taille des données à stocker :
Pour récupérer la taille allouée pour un type, la syntaxe en C est :
taille = sizeof(unsigned short);
En C, on utilisera un des types suivants en fonction de la taille des données à stocker :

`signed char`	$[-2^7; 2^7[$
`unsigned char`	$[0; 2^8[$
`short int`	$[-2^{15}; 2^{15}[$
`unsigned short int`	$[0; 2^{16}[$
`int`	$[-2^{31}; 2^{31}[$
`unsigned int`	$[0; 2^{32}[$
`long int`	$[-2^{63}; 2^{63}[$
`unsigned long int`	$[0; 2^{64}[$

```
printf("char : %d octets\n", sizeof(char));
printf("int : %d octets\n", sizeof(int));
printf("long : %d octets\n", sizeof(long));
printf("double : %d octets\n",
sizeof(double));

char : 1 octets
int : 4 octets
long : 4 octets
double : 8 octets
```

c. *Les types réels*

Les types float, double et long double servent à représenter des nombres en virgule flottante. Ils correspondent aux différentes précisions possibles.

float	32 bits	flottant
double	64 bits	flottant double précision
long double	64 bits	flottant quadruple précision

- Un flottant est représenté par une suite de bits dont le bit de poids fort correspond au signe du nombre. Le champ du milieu correspond à la représentation binaire de l'exposant alors que les bits de poids faible servent à représenter la mantisse.

5) Déclarations des variables

Chaque variable doit avoir un nom et un type. Type nom de variable;

L'instruction de déclaration de variable est comme suit

```
int NBEtudiant;
int note, valeur;
float note, moyenne;
char caractére;
```

6) Les constantes

Une constante est une valeur qui apparaît littéralement dans le code source d'un programme, le type de la constante étant déterminé par la façon dont la constante est écrite.

Elles ne peuvent pas changer de valeur au cours du programme. Avec le commande #define, il est possible de donner un nom symbolique à une constante littérale définies dans les fichiers d'entête ou dans le texte du programme (de préférence au début).

Exemple :

```
/* Les constantes symboliques définies par l'utilisateur */
#define MAX 100
#define PI 3.14
#define LANGAGE "C"
```

Les constantes entières & Réelles :

Par exemple, le taux de conversion euros/francs est fixé à 6,55957, la TVA est une valeur fixe de 20,6.

```
#define TVA 20.6
#define DIV 100
```

```
int main(void){
        printf("10 euros HT = %.2lf TTC\n", 10*TVA/DIV + 10);
        printf("20 euros HT = %.2lf TTC\n", 20*TVA/DIV + 20);
        printf("30 euros HT = %.2lf TTC\n", 30*TVA/DIV + 30);
        return 0;
}
```

Les constantes caractères

Pour désigner un caractère imprimable, il suffit de le mettre entre apostrophes (par ex. 'A' ou '$').

Les seuls caractères imprimables qu'on ne peut pas représenter de cette façon sont l'antislash et l'apostrophe, qui sont respectivement désignés par \\ et \'.

les caractères non-imprimables les plus fréquents disposent aussi d'une notation plus simple :

`\n`	nouvelle ligne	`\r`	retour chariot
`\t`	tabulation horizontale	`\f`	saut de page
`\v`	tabulation verticale	`\a`	signal d'alerte
`\b`	retour arrière		

7) Les opérateurs

L'affectation : variable = expression

- Les opérateurs arithmétiques

+ addition
- soustraction
* multiplication
/ division
% reste de la division (modulo)

- Les opérateurs relationnels

> strictement supérieur
>= supérieur ou égal
< strictement inférieur
<= inférieur ou égal
== égal
!= différent

- Les opérateurs logiques booléens

&& et logique
|| ou logique
! négation logique

- Les opérateurs d'affectation composée

+= -= *= /= %= &= ^= |= <<= >>=
Pour tout opérateur *op, l'expression*
expression-1 op= expression-2

est équivalente à
expression-1 = expression-1 op expression-2

- Incrémentation et de Décrémentation

```
int a = 3, b, c;
b = ++a; /* a et b valent 4 */
c = b++; /* c vaut 4 et b vaut 5 */
```

- L'opérateur de conversion de type, appelé cast, permet de modifier explicitement le type d'un objet. On écrit :

  (*type) objet*

Par exemple :

```
main() {
    int i = 3, j = 2;
    printf("%f \n",(float)i/j);}
```

- L'opérateur adresse

L'opérateur d'adresse & appliqué à une variable retourne l'adresse mémoire de cette variable.
La syntaxe est : &*objet*

8) La fonction principale main

La fonction principale main est la seule à être appelée lors de l'exécution du programme C. Le fichier C contiendra donc toujours une fonction main, et cette fonction retournera toujours la valeur 0 pour indiquer que le programme s'est correctement déroulé.

```
int  main()
{
        déclarations de variables internes
        instructions
}
```

9) Exercice

Ecrire une programme qui permet d'échanger les valeurs de deux variables a et b de type int et float respectivement.

III. Le dialogue avec l'utilisateur

1) La fonction d'écriture printf:

La fonction printf est une fonction d'impression formatée, ce qui signifie que les données sont converties selon le format particulier choisi.

Equivalent à l'instruction ecrire().

Sa syntaxe est :

printf("chaîne de contrôle ",expression-1, ..., expression-n);

format	conversion en	écriture
%d	int	décimale signée
%ld	long int	décimale signée
%u	unsigned int	décimale non signée
%lu	unsigned long int	décimale non signée
%o	unsigned int	octale non signée
%lo	unsigned long int	octale non signée
%x	unsigned int	hexadécimale non signée
%lx	unsigned long int	hexadécimale non signée
%f	double	décimale virgule fixe
%lf	long double	décimale virgule fixe
%e	double	décimale notation exponentielle
%le	long double	décimale notation exponentielle
%g	double	décimale, représentation la plus courte parmi %f et %e
%lg	long double	décimale, représentation la plus courte parmi %lf et %le
%c	unsigned char	caractère
%s	char*	chaîne de caractères

Figure 4 : Les formules d'impression printf

La fonction de saisie scanf

La fonction scanf permet de saisir des données au clavier et de les stocker aux adresses spécifiées par les arguments de la fonctions.

Equivalent à l'instruction lire();

scanf("chaîne de contrôle ",expression-1, ..., expression-n);

format	type d'objet pointé	représentation de la donnée saisie
%d	int	décimale signée
%hd	short int	décimale signée
%ld	long int	décimale signée
%u	unsigned int	décimale non signée
%hu	unsigned short int	décimale non signée
%lu	unsigned long int	décimale non signée
%o	int	octale
%ho	short int	octale
%lo	long int	octale
%x	int	hexadécimale
%hx	short int	hexadécimale
%lx	long int	hexadécimale
%f	float	flottante virgule fixe
%lf	double	flottante virgule fixe
%Lf	long double	flottante virgule fixe
%e	float	flottante notation exponentielle
%le	double	flottante notation exponentielle
%Le	long double	flottante notation exponentielle
%g	float	flottante virgule fixe ou notation exponentielle
%lg	double	flottante virgule fixe ou notation exponentielle
%Lg	long double	flottante virgule fixe ou notation exponentielle
%c	char	caractère
%s	char*	chaîne de caractères

Figure 5 : Les formats de saisie pour scanf

2) Exercices

Exercice 1 :
Traduire l'algorithme suivant en C et expliquer le résultat.
algo affect1b
var a : entier
var b : entier
Début
b ← 5
a ← b+1
b ← 2
Fin

Exercice 2 :
Écrire un programme permettant d'inverser les valeurs de deux variables.
Exercice 3 :

Ecrire un programme qui lit une variable de type entier et qui affiche sa valeur ainsi que son carré.

Exercice 4 :

Ecrire un programme qui demande à l'utilisateur une variable et qui affiche sa table de multiplication

IV. Les instructions Alternatives

1) L'instruction if

Les instructions de contrôle est toute instruction qui permet de contrôler le fonctionnement d'un programme.

Branchement conditionnel if---else :

```
if ( expression-1 )
        instruction-1
else if ( expression-2 )
        instruction-2
...
else if ( expression-n )
        instruction-n
else
        instruction-1
```

2) Branchement multiple switch

Sa forme la plus générale est celle-ci :

```
switch ( expression ) {
case constante-1:
                liste d'instructions 1
                break;
case constante-2:
                liste d'instructions 2
                break;
case constante-n:
                liste d'instructions n
                break;
default:
                liste d'instructions 1
}
```

V. Les instructions répétitives

Les boucles permettent de répéter une série d'instructions tant qu'une certaine condition n'est pas vérifiée.

1) Boucle while :

Sa forme la plus générale est celle-ci :

```
while ( expression )
```

```
        {instructions}
```

2) Boucle do---while

Sa forme la plus générale est celle-ci :

```
Do{
        Instruction}
while ( expression );
```

3) Boucle for

Sa forme la plus générale est celle-ci :

```
for ( expr 1 ; expr 2 ; expr 3)
        Instruction
```

Exemple :

```
for (i = 0; i < 10; i++)
        printf("\n i = %d",i);
```

4) Exercices

Exercice 5 : calcul de remise

A la caisse d'un supermarché, nous bénéficions d'une remise :

- 1% de remise pour un achat compris entre 300 et 750 euros
- 2% au-delà de 750 euros

Exercice 6 : (Estimation du prix de revient d'un véhicule)

Il existe un barème pour l'évaluation du prix de revient kilométrique des véhicules. Écrire un algorithme effectuant le calcul de ce prix en fonction de nb, nombre de kilomètres parcourus. Les Règles :

Nb de km \ puissance fiscale	5cV	6cV
jusqu'à 5000	n1 * 0,43 (= p1)	n * 0,47
de 5001 à 20000	(n2 * 0,23) + p1 (=p2)	(n * 0,27) + 1000
au-delà de 20000	(n3 * 0,28) + p2	n * 0,32

Où n est le nombre total de kilomètres parcourus, n1 le nombre de kilomètres parcourus entre 0 et 5000, n2 le nombre de kilomètres parcourus entre 5001 et 20000 et n3 le nombre de kilomètre parcourus au-delà de 20000.

Exemple : si j'ai parcouru 8500 km, n=8500, n1=5000, n2=3500 et n3=0.

Exercice 7

Écrire un programme qui affiche la somme des n premiers entiers. N étant un nombre saisi par l'utilisateur.

Exercice 8 (Placement d'argent) :

Si l'on place somme en dirhams au 1 janvier de l'année anDepot à taux 2% (en accumulant les intérêts), quelle va être la somme présente sur le compte le 1 janvier de de l'année anRetrait ?